```
    strange.data.d[1] = 32 ;

    printf ( "\n%d", strange.key.i ) ;
    printf ( "\n%d", strange.data.j ) ;
    printf ( "\n%d", strange.key.c[0] ) ;
    printf ( "\n%d", strange.data.d[0] ) ;
    printf ( "\n%d", strange.key.c[1] ) ;
    printf ( "\n%d", strange.data.d[1] ) ;
}
```

And here is the output...

```
512
512
0
0
32
32
```

Just as we do with nested structures, we access the elements of the union in this program using the '.' operator twice. Thus,

```
strange.key.i
```

refers to the variable **i** in the structure **key** in the union **strange**. Analysis of the output of the above program is left to the reader.

Utility of Unions

Suppose we wish to store information about employees in an organization. The items of information are as shown below:

```
Name
Grade
Age
If Grade = HSK  (Highly Skilled)
    hobbie name
    credit card no.
If Grade = SSK (Semi Skilled)
    Vehicle no.
    Distance from Co.
```


Since this is dissimilar information we can gather it together using a structure as shown below:

```
struct employee
{
    char n[ 20 ] ;
    char grade[ 4 ] ;
    int age ;
    char hobby[10] ;
    int crcardno ;
    char vehno[10] ;
    int dist ;
};
struct employee e ;
```

Though grammatically there is nothing wrong with this structure, it suffers from a disadvantage. For any employee, depending upon his grade either the fields hobby and credit card no. or the fields vehicle number and distance would get used. Both sets of fields would never get used. This would lead to wastage of memory with every structure variable that we create, since every structure variable would have all the four field apart from name, grade and age. This can be avoided by creating a **union** between these sets of fields. This is shown below:

```
sruct info1
{
    char hobby[10] ;
    int crcardno ;
};
sruct info2
{
```